

Objects You Already Use

COMSC-1033 Computer Science I

Jeremy Evert

SWOSU

Week 9

Let's begin on familiar ground: a name, a message, or a list of places. Each one is a value in a Python program.

Follow the dot

```
text = "luna"
upper_text = text.upper()
print(upper_text)  # expect: LUNA
print(text)  # expect: luna
print(type(upper_text).__name__)  # expect: str
pieces = text.split("n")
print(pieces)  # expect: ['lu', 'a']
print(type(pieces).__name__)  # expect: list
```

The next clue is a dot.

A string method gives back a value

```
text = "luna"  
upper_text = text.upper()  
print(upper_text)  # expect: LUNA  
print(text)        # expect: luna  
print(type(upper_text).__name__)  # expect: str  
pieces = text.split("n")  
print(pieces)      # expect: ['lu', 'a']  
print(type(pieces).__name__)      # expect: list
```

Strings give us a useful surprise. A string method such as uppercase does not edit the original string in place.

A list can change in place

```
route = ["Terra", "Luna"]
returned = route.append("Sol")
print(route)  # expect: ['Terra', 'Luna', 'Sol']
print(returned)  # expect: None
backup = route.copy()
print(backup)  # expect: ['Terra', 'Luna', 'Sol']
print(type(backup).__name__)  # expect: list
```

Now compare a list. Lists are also objects, but a list is a mutable sequence. Mutable means the list can be changed in place.

Use the return value as evidence

```
text = "luna"
upper_text = text.upper()
print(upper_text) # expect: LUNA
print(text) # expect: luna
print(type(upper_text).__name__) # expect:
    str
pieces = text.split("n")
print(pieces) # expect: ['lu', 'a']
print(type(pieces).__name__) # expect: list
```

```
route = ["Terra", "Luna"]
returned = route.append("Sol")
print(route) # expect: ['Terra', 'Luna', '
    Sol']
print(returned) # expect: None
backup = route.copy()
print(backup) # expect: ['Terra', 'Luna', '
    Sol']
print(type(backup).__name__) # expect: list
```

Let's make the contrast concrete. With a string, call the method and print both the returned value and the original.

Here is a small live debugging session. I start with `'label = " luna "'` and write `'clean = label.strip()'`.

Ask Python what is available

```
signal = "Terra to Luna"
print(type(signal).__name__)  # expect: str
print("upper" in dir(signal)) # expect: True
print(signal.replace("Luna", "Sol")) # expect: Terra to Sol
print(signal.find("Luna")) # expect: 9
print(signal.startswith("Terra")) # expect: True
```

You do not need to memorize every method. Python can help you investigate. Type asks what kind of object you have.

Let's collect the evidence before the final orbit. An object is a value with a type, state, and available behavior.

The circuit is complete. On Terra, we met values we already use.